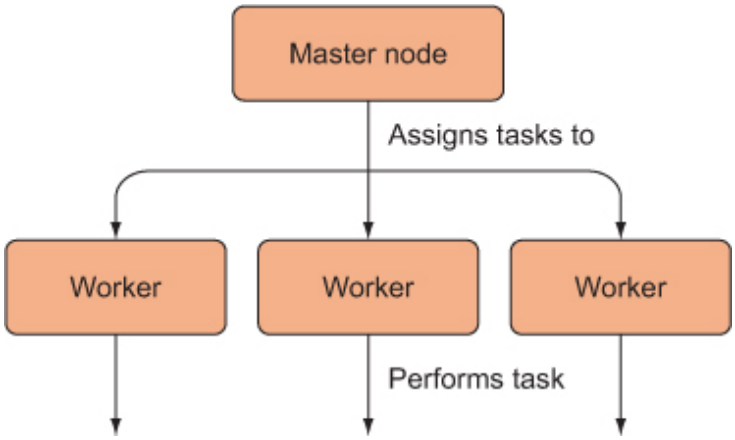
efficiency is a term used to describe how a particular approach improves as more resources are thrown at it and can be calculated as follows:

$$\text{Scaling Efficiency} = \frac{\text{Multiple of Performance Speed up after adding resources}}{\text{Multiple of Resources Added}}$$

In the real world, processes never have a scaling efficiency of 1. There is always some additional cost to adding more machines that decreases efficiency. More realistically, adding 10 more machines will only give us a 9x speedup. Using the previous scaling efficiency equation we can calculate the scaling efficiency as 0.9 (which is pretty good in the real world).

Ultimately we need to combine the results from assessing the fitness of each agent in parallel so that we can recombine and mutate them. Thus, we need to use true parallel processing followed by a period of sequential processing. This is more generally referred to as *distributed computing* (figure 6.14), since we start with a single processor (often called the *master node*) and distribute tasks to multiple processors to run in parallel, and then collect the results back onto the master node.

Figure 6.14. A general schematic for how distributed computing works. A master node assigns tasks to worker nodes; the worker nodes perform those tasks and then send their results back to the master node (not shown).



Every step takes a little bit of network time to communicate between machines, which is something we would not encounter if we were running everything on a single machine. Additionally, if just one machine is slower than the others, the other workers will need to wait. To get the maximal